Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа №1 по дисциплине «Анализ Алгоритмов»

Тема Расстояние Левенштейна и Дамерау — Левенштейна

Студент Платонова М. И.

Группа ИУ7-51Б

Преподаватель Волкова Л. Л.

Москва — 2024 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1 Аналитическая часть **4**

1.1 Расстояние Левенштейна 4

1.2 Формула вычисления расстояния Левенштейна 4

1.3 Расстояние Дамерау — Левенштейна 5

1.4 Формула вычисления расстояния Дамерау — Левенштейна 5

2 Конструкторская часть **7**

2.1 Разработка схем алгоритмов 7

3 Технологическая часть **13**

3.1 Средства реализации 13

3.2 Листинги кода 13

3.3 Функциональные тесты 17

4 Исследовательская часть **18**

4.1 Технические характеристики 18

4.2 Сравнительный анализ времени выполнения алгоритмов 18

4.3 Сравнительный анализ использования памяти алгоритмами 21

4.3.1 Нерекурсивные алгоритмы 21

4.3.2 Рекурсивные алгоритмы 21

4.3.3 Рекурсивные алгоритмы с кэшированием 22

ЗАКЛЮЧЕНИЕ **23**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **24**

ВВЕДЕНИЕ

В данной лабораторной работе будет исследовано расстояние Левенштейна, которое представляет собой минимальное количество операций, требуемых для преобразования одной строки в другую. К таким операциям относятся вставка, удаление и замена символов. Расстояние Левенштейна используется для оценки различий между строками.

Применение расстояния Левенштейна возможно в различных областях:

– корректировка опечаток и ошибок при наборе текста для повышения удобства ввода данных;

– анализ изменений в текстовых файлах, что полезно при работе с системами управления версиями;

– оптимизация поиска схожих строк в больших текстах, используется в поисковых системах.

Целью данной лабораторной работы является исследование и реализация алгоритмов для вычисления расстояния Левенштейна, а также его модификации — Дамерау — Левенштейна. Для достижения этой цели необходимо выполнить следующие задачи:

– исследовать методы расчёта расстояния Левенштейна и Дамерау — Левенштейна;

– реализовать рекурсивный алгоритм для вычисления расстояния Левенштейна;

– реализовать нерекурсивный алгоритм для вычисления расстояния Левенштейна;

– реализовать рекурсивный с кэшем алгоритм для вычисления расстояния Левенштейна;

– реализовать рекурсивный алгоритм для вычисления расстояния Дамерау — Левенштейна;

– провести сравнение эффективности этих алгоритмов, исследуя их затраты по затраченному процессорному времени и памяти;

– сформулировать и представить результаты в отчёте о проведённой лабораторной работе.

1 Аналитическая часть

В данном разделе будут рассмотрены алгоритмы нахождения расстояний Левенштейна и Дамерау — Левенштейна.

1.1 Расстояние Левенштейна

Расстояние Левенштейна [1] — это минимальное количество операций вставки, удаления и замены, необходимых для превращения одной строки в другую.

Каждая из редакторских операций обладает определённой стоимостью (штрафом), которая может изменяться в зависимости от контекста и требований. В данной работе рассматриваются алгоритмы, для которых стоимость всех операций равна 1. Такой подход упрощает анализ и сравнение различных алгоритмов.

- $w(\lambda, \lambda) = 0$ — между пустыми символами (λ обозначает пустую строку);
- $w(a, b) = 1$ — цена замены символа a на b, R (от англ. replace);
- $w(\lambda, b) = 1$ — цена вставки символа b, I (от англ. insert);
- $w(a, \lambda) = 1$ — цена удаления символа a, D (от англ. delete).

Пусть x, y, z — это произвольные символы. Значит, операции, применимые к строкам, состоящим из этих символов, могут быть представлены следующим образом:

- вставка символа: $xy \to xyz$;
- удаление символа: $xyz \to xy$;
- замена символа: $dog \to log$.

1.2 Формула вычисления расстояния Левенштейна

Имеются две строки S_1 и S_2, длины m и n соответственно. Расстояние Левенштейна определяется с помощью рекуррентной формулы (1.1):

$$D(i,j) = \begin{cases} 0, & \text{i = 0, j = 0} \\ i, & \text{j = 0, i > 0} \\ j, & \text{i = 0, j > 0} \\ min \begin{cases} D(i, j-1) + 1 \\ D(i-1, j) + 1 \\ D(i-1, j-1), \text{если } S_1[i] = S_2[j] \\ D(i-1, j-1) + 1, \text{если } S_1[i] \neq S_2[j] \end{cases} & \text{i > 0, j > 0} \end{cases} \tag{1.1}$$

Таким образом, можно последовательно заполнить матрицу, где каждое значение в ячейках с индексами i и j отражает расстояние Левенштейна между подстроками $S_1[:\ i]$ и $S_2[:\ j]$. Итоговое значение расстояния Левенштейна для строк S_1 и S_2 будет расположено в ячейке $D(m, n)$.

1.3 Расстояние Дамерау — Левенштейна

Расстояние Дамерау — Левенштейна определяется как минимальное количество редакторских операций, необходимых для преобразования одной строки в другую. В дополнение к операциям вставки, удаления и замены, здесь также учитывается операция транспозиции, которая позволяет менять местами две соседние буквы, например, метр $\rightarrow$ мерт.

1.4 Формула вычисления расстояния Дамерау — Левенштейна

Имеются две строки S_1 и S_2, длины m и n соответственно. Расстояние Дамерау — Левенштейна рассчитывается по следующей рекуррентной формуле (1.2):

$$D(i,j) = \begin{cases} 0, & j = 0, i = 0 \\ i, & i = 0, i > 0 \\ j, & j > 0, i = 0 \\ min \begin{cases} D(i,j-1)+1, \\ D(i-1,j)+1, \\ D(i-1,j-1), \\ \qquad \text{если } S_1[i] = S_2[j], \\ D(i-1,j-1)+1, \\ \qquad \text{если } S_1[i] \neq S_2[j], \\ D(i-2,j-2), \\ \qquad \text{если } S_1[i] = S_2[j], \\ D(i-2,j-2)+1, \\ \qquad \text{если } S_1[i] \neq S_2[j], \end{cases} & \begin{array}{l} \text{если i > 1, j > 1} \\ S_1[i] = S_2[j-1] \\ S_1[j] = S_2[i-1] \end{array} \\ min \begin{cases} D(i,j-1)+1, \\ D(i-1,j)+1, \\ D(i-1,j-1), \\ \qquad \text{если } S_1[i] = S_2[j], \\ D(i-1,j-1)+1, \\ \qquad \text{если } S_1[i] \neq S_2[j], \end{cases} & \text{иначе} \end{cases} \tag{1.2}$$

В результате выполнения поэтапного заполнения матрицы каждая ячейка с координатами i и j будет содержать значение расстояния Дамерау — Левенштейна для подстрок $S_1[:\ i]$

и $S_2[:\ j]$.Итоговое значение расстояния Дамерау — Левенштейна для строк S_1 и S_2 будет расположено в ячейке $D(m, n)$.

Вывод

В данном разделе были рассмотрены расстояния Левенштейна и Дамерау — Левенштейна. Для вычисления этих расстояний представлены формулы, которые могут быть реализованы как рекурсивными, так и нерекурсивными алгоритмами.

2 Конструкторская часть

В данном разделе будут представлены схемы алгоритмов для вычисления расстояний Левенштейна и Дамерау — Левенштейна.

2.1 Разработка схем алгоритмов

На вход алгоритмов поступают строки S_1 и S_2, включающие как русские, так и английские буквы. Результатом работы алгоритмов является целое число, соответствующее искомому расстоянию. Рисунки 2.1 — 2.4 представляют схемы рекурсивных и нерекурсивных алгоритмов для вычисления расстояний Левенштейна и Дамерау — Левенштейна.

Рисунок 2.1 — Нерекурсивный алгоритм нахождения расстояния Левенштейна

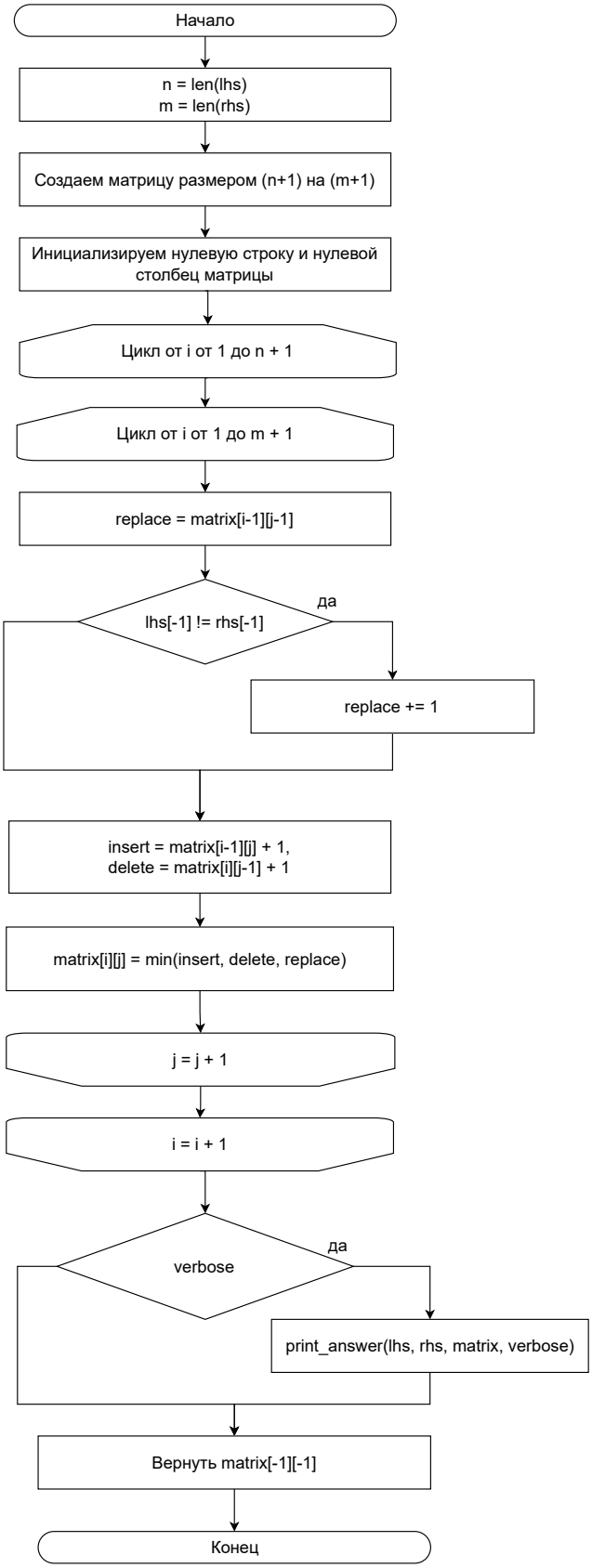


Рисунок 2.2 — Рекурсивный алгоритм нахождения расстояния Левенштейна

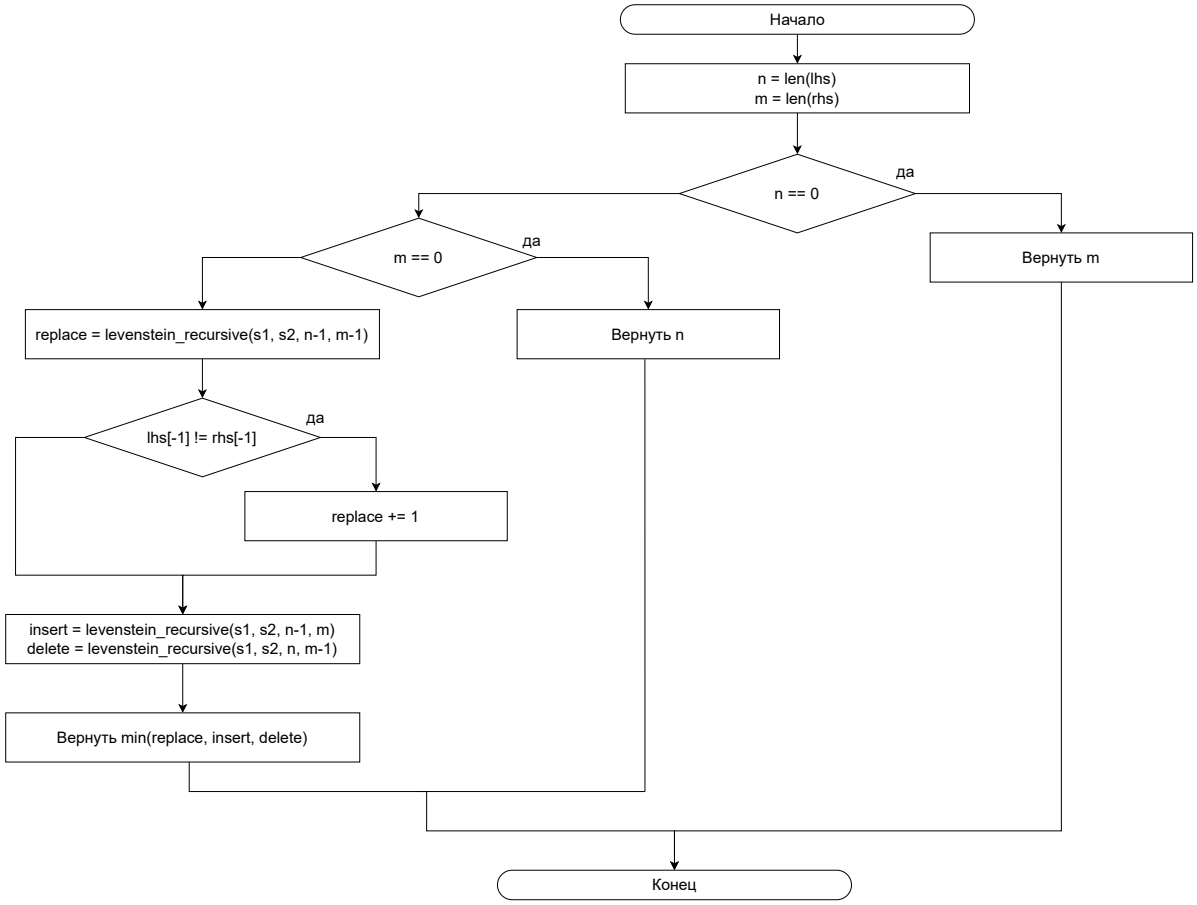


Рисунок 2.3 — Рекурсивный алгоритм с кэшем нахождения расстояния Левенштейна

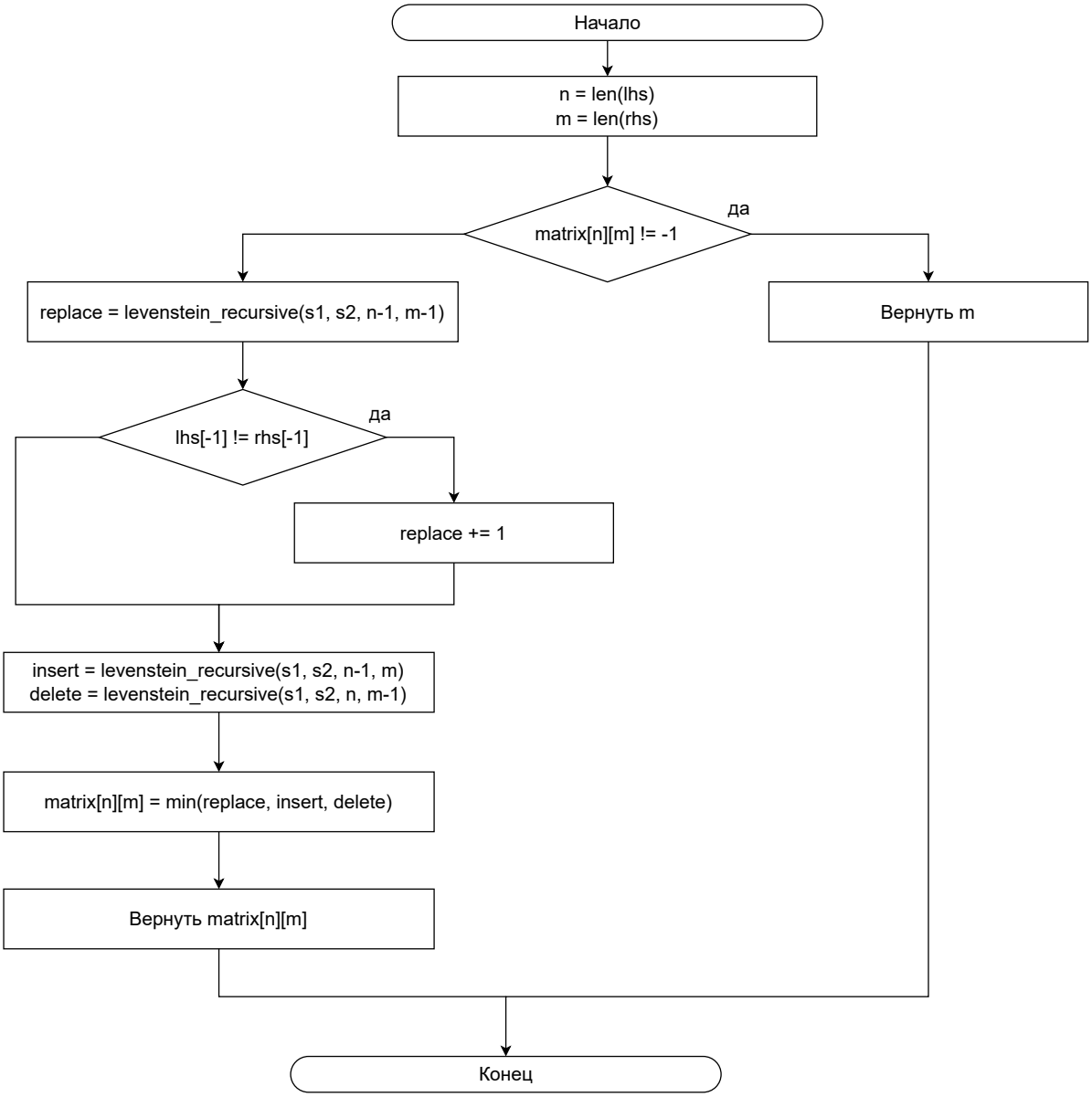
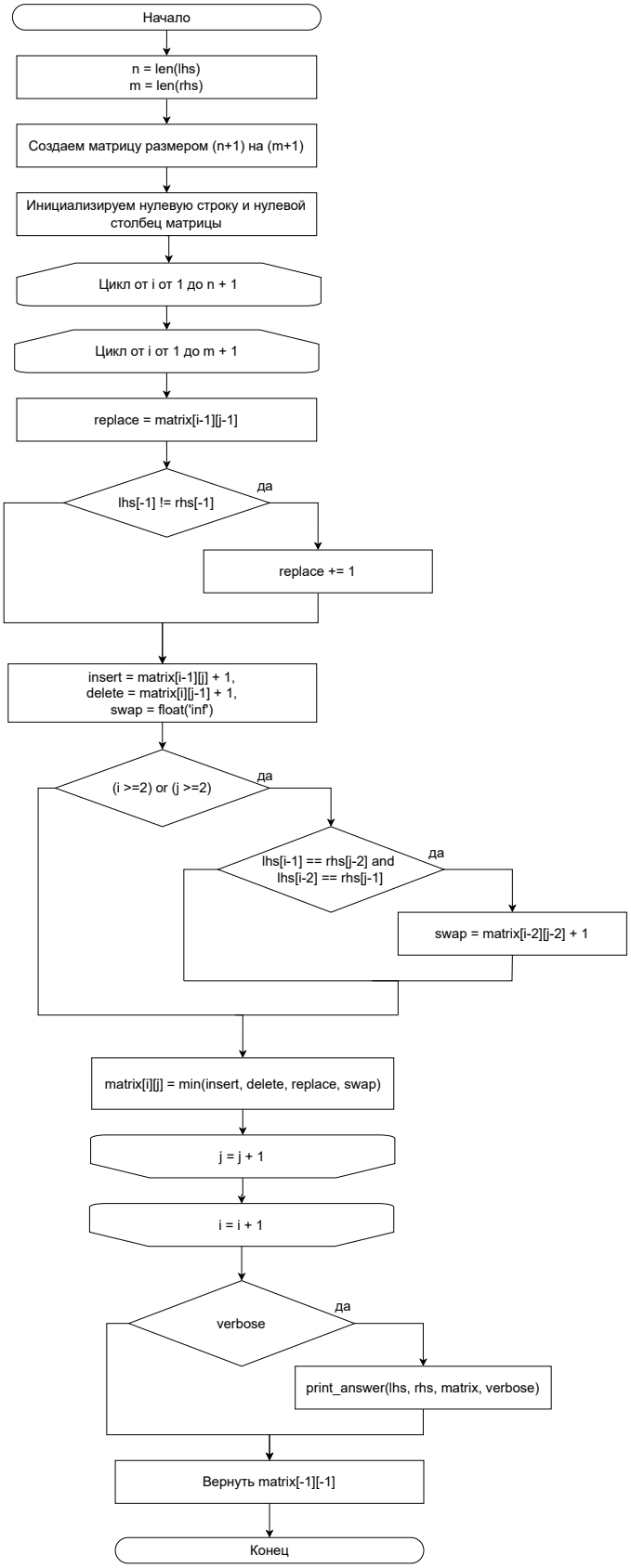


Рисунок 2.4 — Нерекурсивный алгоритм нахождения расстояния Дамерау — Левенштейна



Вывод

В данном разделе были разработаны алгоритмы для нерекурсивного, рекурсивного и рекурсивного с кэшем поиска расстояния Левенштейна, а также нерекурсивного алгоритма поиска расстояния Дамерау — Левенштейна.

3 Технологическая часть

В данном разделе будут приведены данные о выбранном языке программирования, листинги кода и тесты для каждого алгоритма.

3.1 Средства реализации

В данной работе для реализации был выбран язык программирования `Python` [2]. Выбор обусловлен наличием компилятора `MicroPython`, который позволяет создавать программы для работы с микроконтроллером. Также язык включает в себя модуль `time` [3], который предоставляет метод, позволяющий получать процессорное время в тактовых циклах.

3.2 Листинги кода

Листинг 3.1 — Реализация создания матрицы для расстояния Левенштейна

```
def create_matrix(n: int, m: int):
    matrix = [[0 for _ in range(m)] for _ in range(n)]
    for i in range(n):
        for j in range(m):
            if i == 0:
                matrix[i][j] = j
            elif j == 0:
                matrix[i][j] = i
            else:
                matrix[i][j] = -1
    return matrix
```

Листинг 3.2 — Реализация вывода матрицы расстояний

```
def print_answer(lhs: str, rhs: str, matrix, verbose=True):
    if not verbose:
        return

    print("   " + "  ".join(f"{ch}" for ch in rhs))

    for i, row in enumerate(matrix):
        if i == 0:
            print("  " + " ".join(f"{val:2}" for val in row))
        else:
            print(f"{lhs[i - 1]} " + " ".join(f"{val:2}" for val in row
                ))
```

Листинг 3.3 — Реализация матричного нахождения расстояния Левенштейна

```
def levenstein_matrix(lhs: str, rhs: str, verbose=False):
    n = len(lhs)
    m = len(rhs)

    matrix = create_matrix(n + 1, m + 1)

    for i in range(1, n + 1):
        for j in range(1, m + 1):
            replace = matrix[i - 1][j - 1]
            if lhs[i - 1] != rhs[j - 1]:
                replace += 1
            insert = matrix[i - 1][j] + 1
            delete = matrix[i][j - 1] + 1

            matrix[i][j] = min(insert, delete, replace)

    if verbose:
        print_answer(lhs, rhs, matrix, verbose)

    return matrix[-1][-1]
```

Листинг 3.4 — Реализация рекурсивного нахождения расстояния Левенштейна

```
def levenstein_recursive(lhs: str, rhs: str):
    n = len(lhs)
    m = len(rhs)

    if n == 0:
        return m
    if m == 0:
        return n

    replace = levenstein_recursive(lhs[:-1], rhs[:-1])

    if lhs[-1] != rhs[-1]:
        replace += 1

    insert = levenstein_recursive(lhs[:-1], rhs) + 1
    delete = levenstein_recursive(lhs, rhs[:-1]) + 1

    return min(replace, insert, delete)
```

Листинг 3.5 — Реализация рекурсивного нахождения расстояния Левенштейна с кэшированием

```
def levenstein_recursive_cash(lhs: str, rhs: str, matrix):
    n = len(lhs)
    m = len(rhs)
    if matrix[n][m] != -1:
        return matrix[n][m]

    replace = levenstein_recursive_cash(lhs[:-1], rhs[:-1], matrix)

    if (lhs[-1] != rhs[-1]):
        replace += 1

    insert = levenstein_recursive_cash(lhs[:-1], rhs, matrix) + 1
    delete = levenstein_recursive_cash(lhs, rhs[:-1], matrix) + 1

    matrix[n][m] = min(replace, insert, delete)
    return matrix[n][m]
```

Листинг 3.6 — Реализация нахождения расстояния Левенштейна с кэшированием

```
def levenstein_cash(lhs: str, rhs: str):
    n = len(lhs)
    m = len(rhs)
    matrix = create_matrix(n + 1, m + 1)
    return levenstein_recursive_cash(lhs, rhs, matrix)
```

Листинг 3.7 — Реализация матричного нахождения расстояния Дамерау — Левенштейна

```
def damerau_levenstein_matrix(lhs: str, rhs: str, verbose=False):
    n = len(lhs)
    m = len(rhs)

    matrix = create_matrix(n + 1, m + 1)

    for i in range(1, n + 1):
        for j in range(1, m + 1):
            replace = matrix[i - 1][j - 1]
            if lhs[i - 1] != rhs[j - 1]:
                replace += 1
            insert = matrix[i - 1][j] + 1
            delete = matrix[i][j - 1] + 1
            swap = float('inf')

            if i >= 2 and j >= 2:
                if lhs[i - 1] == rhs[j - 2] and lhs[i - 2] == rhs[j -
                   1]:
                    swap = matrix[i - 2][j - 2] + 1

            matrix[i][j] = min(insert, delete, replace, swap)

    if verbose:
        print_answer(lhs, rhs, matrix, verbose)

    return matrix[-1][-1]
```

3.3 Функциональные тесты

В таблице 3.1 приведены результаты функционального тестирования для алгоритмов, реализующих вычисление расстояний Левенштейна и Дамерау — Левенштейна. Все тесты пройдены успешно.

Таблица 3.1 — Функциональные тесты

1-я строка	2-я строка	Результаты				
		Ожид.р.	Нер.Л.	Рек.Л.	Р.х.Л.	Нер.Д-Л.
[]	[]	0	0	0	0	0
[]	world	5	5	5	5	5
orange	oarnge	2	2	2	2	1
Paris	Pariss	2	2	2	2	1
garden	gaden	1	1	1	1	1
sister	sistar	1	1	1	1	1
music	muesic	1	1	1	1	1
programming	programing	2	2	2	2	1
beautiful	beutiful	3	3	3	3	3
intelligent	intellignet	5	5	5	5	5

Вывод

В данном разделе разработаны и приведены листинги кода алгоритмов вычисления расстояний Левенштейна и Дамерау — Левенштейна, также представлены функциональные тесты.

4 Исследовательская часть

В данном разделе будут представлены результаты исследования эффективности работы алгоритмов по времени выполнения.

4.1 Технические характеристики

Технические характеристики устройства, на котором проводились замеры времени.

– Микроконтроллер STM32F767ZIT6 с 32-битным ядром ARM Cortex-M7, работающим на частоте 216 МГц, с 2 Мбайт Flash-памяти и 512 Кбайт оперативной памяти (RAM).

Так как на данной плате отсутствует пользовательский режим, все замеры времени выполнения программы отражают процессорное время. Это позволяет более точно оценить производительность алгоритмов.

4.2 Сравнительный анализ времени выполнения алгоритмов

В таблице 4.1 показаны результаты измерения времени работы алгоритмов для вычисления всех расстояний Левенштейна.

Таблица 4.1 — Результаты измерений времени выполнения алгоритмов (в тактах)

max width=

Длина строки	Нерек. Лев.	Рек. Лев.	Лев. с кэш	Дамерау – Лев.
0	15207.77	11000.50	11170.68	15221.92
1	25276.51	3024.89	33784.20	25874.56
2	42310.99	62566.82	101735.29	42891.34
3	77849.65	330447.73	261910.67	78800.58
4	117412.66	1773930.79	504072.48	119643.49
5	158795.33	10494688.47	824528.83	161234.76
10	578839.42	-	1345000.00	755231.39
15	1256945.93	-	8478098.41	1308764.82
20	2196200.39	-	18426500.50	3157063.74
25	3282153.25	-	-	3324152.68
30	4813620.04	-	-	7033601.76
40	8667289.41	-	-	12942068.41
50	13789404.01	-	-	20422356.67

В результате тестирования на рисунке 4.1 рекурсивный алгоритм Левенштейна оказался наименее эффективным по времени выполнения, в то время как нерекурсивный алгоритм показал наилучшие результаты. Далее приведены графики сравнения оставшихся алгоритмов на большей длине строк.

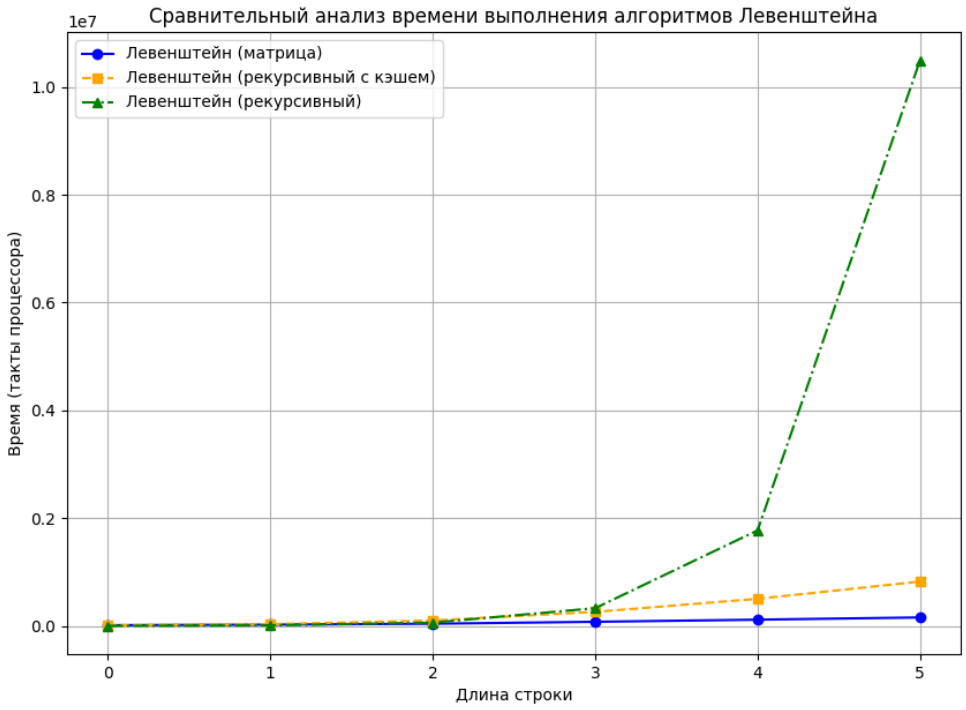


Рисунок 4.1 — Сравнение времени всех реализаций алгоритмов нахождения расстояний Левенштейна.

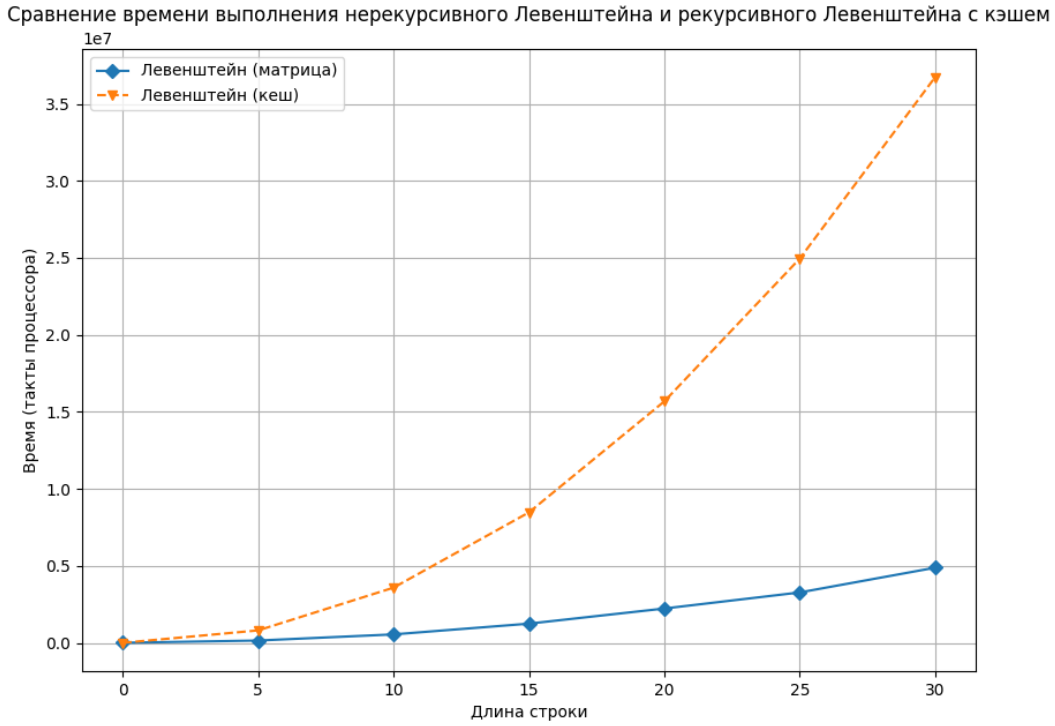


Рисунок 4.2 — Сравнение нерекурсивного алгоритма Левенштейна и рекурсивного алгоритма Левенштейна с использованием кэша по времени.

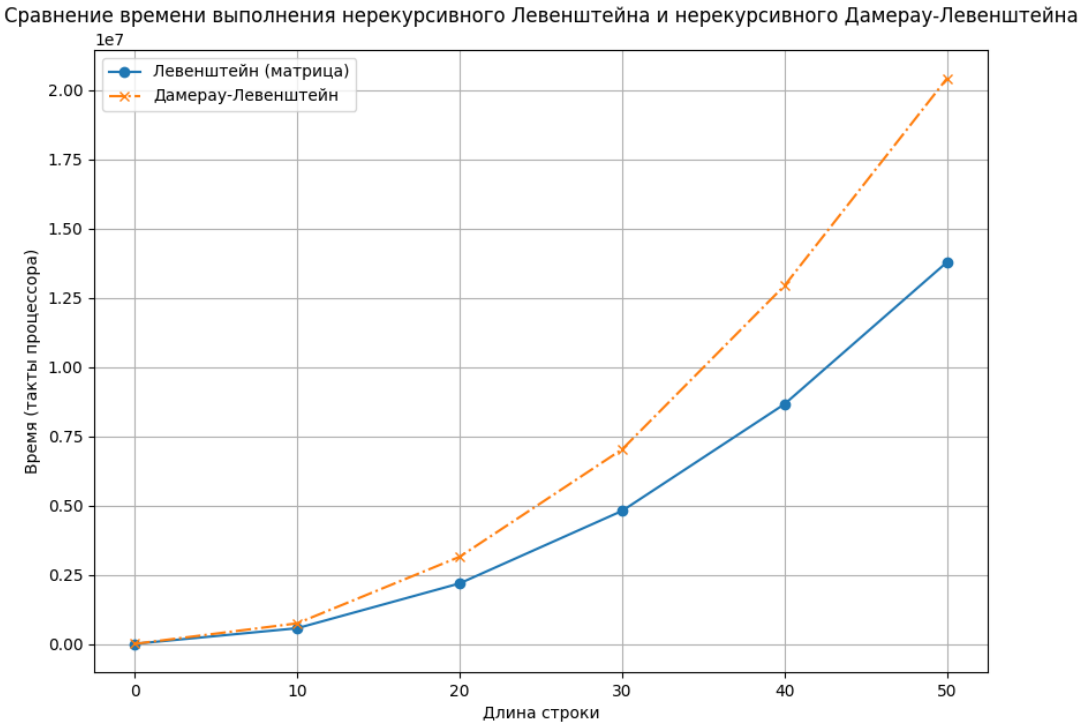


Рисунок 4.3 — Сравнение нерекурсивного алгоритма Левенштейна и нерекурсивного алгоритма Дамерау — Левенштейна по времени.

4.3 Сравнительный анализ использования памяти алгоритмами

4.3.1 Нерекурсивные алгоритмы

Нерекурсивные алгоритмы нахождения расстояния `Левенштейна` [4] и Дамерау — Левенштейна имеют одинаковые затраты по памяти, так как оба используют матрицу для хранения промежуточных результатов. В реализации `levenstein_matrix` и `damerau_levenstein_matrix` память необходима для создания матрицы, размер которой определяется длинами строк n и m. Каждая ячейка матрицы содержит целочисленное значение, и общий объем памяти, необходимый для её хранения, пропорционален $(n+1) \times (m+1)$, где n и m — длины строк.

Кроме хранения значений матрицы, связанных с операциями вставки, удаления, замены и перестановки символов, используются дополнительные переменные.

Затраты памяти для нерекурсивных алгоритмов:

$$M_{\text{iter}} = (n+1) \times (m+1) \times \text{size(int)} + (4 \times \text{size(int)})$$

где:

- n и m — длины строк;
- size(int) — объем памяти для хранения одного целочисленного значения;
- $4 \times$ size(int) — затраты на хранение четырёх переменных, необходимых для операций вставки, удаления, замены и перестановки символов.

4.3.2 Рекурсивные алгоритмы

Для рекурсивных алгоритмов, таких как `levenstein_recursive`, память расходуется иначе. Каждый раз, когда вызывается рекурсивная функция, она занимает место в стеке вызовов. Глубина стека, которая влияет на использование памяти, зависит от длин строк n и m. В худшем случае глубина стека равна $n+m$, это означает, что память будет расходоваться на хранение локальных переменных каждой рекурсивной функции.

Затраты памяти для рекурсивного алгоритма:

$$M_{\text{rec}} = (n+m) \times \text{size(call stack)} + (3 \times \text{size(int)})$$

где:

- size(call stack) — объем памяти, используемой на каждый рекурсивный вызов;
- $3 \times$ size(int) — затраты на промежуточные переменные для вставки, удаления и замены символов.

4.3.3 Рекурсивные алгоритмы с кэшированием

Рекурсивные алгоритмы с кэшированием делают процесс вычисления расстояния Левенштейна более эффективным, уменьшая количество рекурсивных вызовов за счёт сохранения уже вычисленных результатов. Хотя такие алгоритмы требуют больше памяти, чем обычные рекурсивные алгоритмы, из-за необходимости хранения кэша, они ускоряют выполнение за счёт сокращения времени, необходимого для повторного вычисления одних и тех же значений.

$$M_{\text{rec_cache}} = (n+1) \times (m+1) \times \text{size(int)} + (n+m) \times \text{size(call stack)} + (3 \times \text{size(int)})$$

- $(n+1) \times (m+1) \times$ size(int) – затраты на хранение матрицы;
- $(n+m) \times$ size(call stack) – затраты на стек вызовов. Глубина стека $n+m$, так как в худшем случае рекурсивный процесс обрабатывает все подстроки длиной n и m;
- $3 \times$ size(int) – затраты на промежуточные переменные для вставки, удаления и замены.

Таким образом, затраты памяти для рекурсивного алгоритма с кэшированием включают в себя: память на хранение кэша, память для стека рекурсивных вызовов, которая уменьшается благодаря кэшированию, и постоянные затраты на хранение трёх промежуточных переменных.

Вывод

Нерекурсивные алгоритмы обеспечивают эффективное использование памяти благодаря сохранению матрицы промежуточных результатов, что позволяет избежать расходов на стек вызовов. Эти алгоритмы предпочтительнее при работе с длинными строками, однако их требования к памяти высоки из-за размеров матрицы.

Рекурсивные алгоритмы без кэширования более экономичны на этапе промежуточных вычислений, но затраты на стек вызовов возрастают с увеличением длины строк, что снижает их эффективность для больших входных данных.

Рекурсивные алгоритмы с кэшированием требуют больше памяти из-за необходимости хранения кэша вычисленных результатов, поэтому нерекурсивные алгоритмы остаются более эффективными в плане использования памяти.

ЗАКЛЮЧЕНИЕ

Цель работы достигнута. В ходе выполнения данной лабораторной работы были решены следующие задачи:

– исследованы методы расчёта расстояния Левенштейна и Дамерау — Левенштейна;

– реализован рекурсивный алгоритм для вычисления расстояния Левенштейна;

– реализован нерекурсивный алгоритм для вычисления расстояния Левенштейна;

– реализован рекурсивный с кэшем алгоритм для вычисления расстояния Левенштейна;

– реализован рекурсивный алгоритм для вычисления расстояния Дамерау — Левенштейна;

– проведено сравнение эффективности этих алгоритмов, исследуя их затраты по затраченному процессорному времени и памяти;

– представлены полученные результаты в виде отчёта о выполненной лабораторной работе.

Исследования показали, что нерекурсивные алгоритмы быстрее рекурсивных и требуют меньше памяти.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Левенштейн, В. И. Бинарные коды, исправляющие удаления, вставки и перестановки. Доклады Академии Наук СССР, 1966, т. 163, с. 845–848.

2. Python Software Foundation. Официальная документация языка Python [Электронный ресурс]. – Режим доступа: `https://www.python.org/doc/` (дата обращения: 29.09.2024).

3. Python Software Foundation. Документация модуля time – Доступ ко времени и его преобразования [Электронный ресурс]. – Режим доступа: `https://docs.python.org/3/library/time.html` (дата обращения: 29.09.2024).

4. Чёрненький В. М., Гапанюк Ю. Е. Оценка точности измерительных систем // Вестник МГТУ им. Н.Э. Баумана. Сер. Приборостроение. 2012. № 3. С. 102–110. ISSN 0236-3933.